شرح مشروع هامش الربح (Qafila)

ما المشكلة، وماذا بنينا، ولماذا كل قرار، وكيف نتأكد أن الأرقام صحيحة

1. الفكرة في دقيقة

قافلة تبيع مواد غذائية ومنزلية أونلاين. الفريق المالي يرى **الإيراد** (ما دفعه العملاء) لكنه لا يرى **هامش الربح الإجمالي**، أي ما يتبقى بعد دفع ثمن البضاعة للموردين. تكلفة البضاعة موجودة في ملفات أسعار يرسلها الموردون كل أسبوع، وهذه الملفات غير نظيفة، وتتغير أسعارها مع الوقت، وأحدها صُحّح متأخراً.

الناتج المطلوب: جدول واحد موثوق اسمه `weekly_margin` يقول لكل **أسبوع × مدينة × تصنيف**: كم بعنا، وكم كلّفتنا البضاعة، وكم ربحنا. والأهم: أن يبقى الجواب صحيحاً **عند إعادة التشغيل** وعند **وصول تصحيح متأخر**، مع القدرة على معرفة «ما الرقم الذي كنا نراه قبل التصحيح؟».

2. مصادر البيانات

المصدر	المكان	ما نأخذ منه
قاعدة بيانات التطبيق (SQLite)	`data/qafila_oltp.sqlite`	جداول `orders` (45,766 أوردر) و`order_items` (172,545 سطر) و`products` (356 منتج) و`stores` (12 فرع)
ملفات أسعار الموردين	`data/suppliers/prices_2026-W36.csv` ... `W39.csv`	ملف لكل أسبوع، يذكر المنتجات التي سُعّرت في ذلك الأسبوع فقط (حوالي 160 منتجاً من 350)
التصحيح المتأخر	`prices_2026-W39_fix.csv`	يصل لاحقاً: يصلّح التكاليف الصفرية ويعدّل 5 أسعار (9 صفوف)

3. الصورة الكاملة

```
files (SQLite + CSV)
   |  Python: ingest()          load everything as-is
   v
DuckDB  schema raw              orders, order_items, products, stores, supplier_prices
   |  dbt staging (views)       parse costs, trim SKUs, Riyadh order date, line revenue
   v
dbt quality.price_quarantine    supplier rows we refuse to use (+ why)
dbt silver.product_cost_history SCD2: one row per SKU per cost version
dbt gold.weekly_margin          each order line costed at the price valid on its order date
   |  Python: publish()         create / skip if unchanged / update in place
   v
Apache Iceberg  (3 tables)      snapshots keep the history
```

الفكرة وراء الطبقات: كل طبقة تفعل شيئاً واحداً ويمكن فحصها وحدها. **raw** يحفظ ما وصل كما هو، و**staging** ينظّف، و**silver** يبني التاريخ، و**gold** يجيب عن سؤال العمل.

4. كل طبقة بالتفصيل

4.1 الهبوط (Land): `ingest()` في `run.py`

- يقرأ الجداول الأربعة من SQLite بمكتبة Python القياسية `sqlite3` (لا إضافات ولا تنزيل من الإنترنت) ويكتبها في DuckDB تحت `raw` schema.
- يقرأ كل ملف `prices_*.csv` **كنصوص بلا تحويل**، ويضيف لكل صف اسم الملف (`source_file`) ورقم الصف (`file_row`). ويقرأ بترميز `utf-8-sig` لأن ملف W38 يبدأ بعلامة BOM تلتصق باسم أول عمود لو لم تُزَل. ولأن الملفات تختلف في الأعمدة (W39 غيّر اسم عمود وأضاف آخر) يأخذ اتحاد الأعمدة ويملأ الفراغ بـ `NULL`.
- **إعادة التشغيل آمنة:** كل جدول raw يُعاد بناؤه (`CREATE OR REPLACE`) من الملفات في كل مرة، فلا يتكرر أي صف أبداً، ويعكس raw دائماً ما في مجلد البيانات الآن.

4.2 التنظيف (Staging): 5 موديلات (views)

الموديل	ماذا يفعل
`stg_products`	`trim` للـ SKU (منتجان في القاعدة لهما مسافات في آخر الـ SKU)
`stg_stores`	الفرع والمدينة
`stg_orders`	يحوّل `created_at` (UTC نصاً) إلى تاريخ الأوردر بتوقيت الرياض: الوقت + 3 ساعات ثم نأخذ اليوم
`stg_order_items`	يحسب إيراد السطر: `round(qty × unit_price − discount, 2)`
`stg_supplier_prices`	يحوّل التكلفة النصية إلى رقم، ويجعل الـ SKU نظيفاً والتاريخ تاريخاً. **لا يحذف شيئاً**: الصفوف السيئة تُصنَّف لاحقاً

تحويل التكلفة هو أهم جزء هنا. العمود اسمه `unit_cost` حتى W38 و`cost_sar` من W39، فنأخذ أيهما موجود (`coalesce`). ثم نوحّد الشكل:

ما في الملف	المشكلة	الناتج
`7.71`	سليم	7.71
`9,49`	فاصلة عشرية	9.49
`٩٫٤٩`	أرقام عربية-هندية وفاصل عربي	9.49
`-5.5` أو `0`	قيمة غير موجبة	رقم، لكن يُحجَر لاحقاً
نص غير رقمي	غير قابل للتحويل	`NULL` (يُحجَر)

ولا يمكن استخدام `cast` العادي لأنه ينهار عند أول قيمة سيئة؛ نستخدم `try_cast` الذي يعطي `NULL` بدلاً من الانهيار.

4.3 الحجر: `quality.price_quarantine`

صف المورد يُرفض بترتيب ثابت: أولاً إن كان الـ SKU **غير موجود** في `products` فالسبب `unknown_sku`، وإلا إن كانت التكلفة **ليست رقماً موجباً** فالسبب `invalid_cost`. نحفظ التكلفة **كما كُتبت في الملف** (`raw_cost`) حتى يستطيع من يراجع أن يرى القيمة الأصلية. الصفوف المحجورة تُتجاهل في كل مكان آخر. في بيانات التجربة: 4 صفوف `invalid_cost` و3 `unknown_sku`، كلها في W39.

4.4 التاريخ: (SCD2) `silver.product_cost_history`

SCD2 معناه: بدلاً من تخزين «سعر المنتج الآن»، نخزن **كل سعر مرّ عليه ومتى كان ساريًا**. لكل نسخة: `valid_from` (أول يوم يسري فيه، شامل) و`valid_to` (أول يوم لا يسري فيه، أي **غير شامل**، وفارغ للنسخة الحالية) و`is_current`. الخطوات الأربع:

1. **المقبول:** صفوف الموردين غير المحجورة.
2. **الملف الأخير يفوز:** إن تكرر (SKU, `valid_from`) في أكثر من ملف نأخذ الصف من الملف الذي يأتي **اسمه أخيراً** بالترتيب الأبجدي. لذلك يحلّ `prices_2026-W39_fix.csv` محل `prices_2026-W39.csv` تلقائياً دون أي كود خاص بالأسماء.
3. **نسخة جديدة عند التغيّر فقط:** نقارن كل تكلفة بتكلفة الصف السابق للـ SKU نفسه (`lag`) ونحتفظ بالصف فقط إن اختلفت. وهذا يحلّ مشكلتين: المنتج الغائب عن ملف أسبوع يبقى على سعره السابق (لا صف جديد له أصلاً)، والسعر المكرر دون تغيير لا يفتح نسخة جديدة.
4. **نهاية النسخة:** `valid_to` = بداية النسخة التالية (`lead`)، وفارغة للأخيرة.

مثال حقيقي: المنتج `QF-BAK-1058`

ما قالته الملفات	السعر المكتوب
W36 (`2026-08-31`)	`4.16`
W37	المنتج غير مذكور في الملف (يبقى 4.16)
W38 (`2026-09-14`)	`3,87` (فاصلة عشرية)
W39 (`2026-09-21`)	`3.58`، ثم صُحّح في ملف التصحيح إلى `3.45`

sku	unit_cost_sar	valid_from	valid_to	is_current
QF-BAK-1058	4.16	2026-08-31	2026-09-14	False
QF-BAK-1058	3.87	2026-09-14	2026-09-21	False
QF-BAK-1058	3.45 (كانت 3.58 قبل التصحيح)	2026-09-21	فارغ	True

4.5 الهامش: `gold.weekly_margin`

أي سطر أوردر يُحسب؟ إذا كان الأوردر `delivered` والكمية أكبر من 0 وتاريخه (بتوقيت الرياض) في `2026-08-31` أو بعده (أول أسبوع سعّره الموردون). تاريخ البدء هذا هو القيمة الوحيدة المكتوبة ثابتة، وهي متغيّر واحد في `dbt_project.yml`.

كيف يجد السطر تكلفته؟ بربط زمني (point-in-time join) مع الـ SCD2: نأخذ النسخة التي `valid_from ≤ تاريخ الأوردر < valid_to`. لأن نسخ المنتج الواحد لا تتداخل أبداً، يطابق السطر نسخة واحدة على الأكثر، فلا يتكرر. والسطر الذي ليس له نسخة صالحة (أوردر قبل أول سعر للمنتج) يسمى **غير مُكلَّف**: يُحسب في `lines` و`revenue_sar` لكن لا يدخل في أعمدة التكلفة.

مثال حقيقي: سطر واحد

- الأوردر `5030696` السطر 4 من `QF-BAK-1058`، حالته `delivered`.
- وقته 09:39 UTC يوم `2026-09-12`، فبتوقيت الرياض 12:39 من اليوم نفسه، أي أسبوع ISO رقم 37.
- الإيراد = `round(1 × 5.41 − 0.54, 2) = 4.87`.
- النسخة السارية يوم `2026-09-12` هي 4.16 (من `2026-08-31` إلى `2026-09-14`)، فالتكلفة 4.16 والهامش `4.87 − 4.16 = 0.71`.
- يُجمَع في صف الأسبوع 37 / الدمام / المخبوزات (Bakery).

كيف تقرأ جدول `weekly_margin`

العمود	المعنى
`lines`	عدد أسطر الأوردرات المحسوبة
`revenue_sar`	إيراد كل الأسطر (المُكلَّفة وغير المُكلَّفة)
`costed_revenue_sar`	إيراد الأسطر التي لها تكلفة فقط
`cost_sar`	مجموع `الكمية × تكلفة الوحدة` للأسطر المُكلَّفة
`margin_sar`	`costed_revenue_sar − cost_sar`. وهو الهامش الحقيقي للأسطر المعروفة التكلفة فقط
`uncosted_lines`	عدد الأسطر بلا تكلفة. كلما زاد، قلّت دلالة `margin_sar`

مثال صف من الناتج: الأسبوع 36 / الدمام / Bakery: `lines=179`، `revenue=1739.23`، `cost=544.94`، `margin=246.33`، `uncosted_lines=110`، `costed_revenue=791.27`. أي أن 110 من 179 سطراً لا تعرف تكلفتها بعد، فالهامش 246.33 يخص الأسطر الـ 69 المكلّفة فقط.

> نسبة الأسطر غير المُكلَّفة مرتفعة (20,626 من 82,404) وهذا متوقع: كل ملف أسبوعي يسعّر حوالي 160 منتجاً من 350، والأوردرات التي تسبق أول سعر للمنتج لا تملك تكلفة وفق القاعدة 9. المشروع لا يخمّن تكلفة ليست موجودة.

4.6 النشر إلى Iceberg: `publish()`

- **أول مرة:** يُنشأ الجدول ويُملأ (snapshot رقم 1).
- **نفس البيانات مرة ثانية:** يقارن الصفوف الجديدة بما في الجدول؛ إن تطابقت لا يكتب شيئاً ولا يضيف snapshot («لا يغيّر شيئاً»).
- **بيانات تغيّرت** (مثل وصول التصحيح): يستبدل صفوف الجدول **في مكانه** بـ snapshot واحد جديد (حذف الملفات القديمة وإضافة الجديد في commit واحد). لا يُحذف الجدول ولا يُعاد إنشاؤه، فتبقى الـ snapshots

القديمة قابلة للقراءة.

هذا هو «السفر عبر الزمن»: `table.scan(snapshot_id=…)` يعرض أرقام الهامش كما كانت قبل التصحيح. على بيانات التجربة:

الحالة	إجمالي التكلفة (ر.س)	إجمالي الهامش (ر.س)
قبل التصحيح (snapshot رقم 1)	1,654,199.98	801,016.88
بعد التصحيح (snapshot رقم 2)	1,651,701.06	803,515.80

5. القواعد العشر وأين تتحقق

#	القاعدة	أين في الكود
1	التكلفة من `unit_cost` أو `cost_sar`، وقراءة الفاصلة والأرقام العربية	`stg_supplier_prices.sql`
2	مقارنة الـ SKU بعد `trim` من الجهتين	`stg_products.sql` و`stg_supplier_prices.sql`
3	الحجر: `unknown_sku` ثم `invalid_cost`	`price_quarantine.sql`
4	المنتج الغائب يحتفظ بتكلفته السابقة	`product_cost_history.sql` (لا صف جديد إلا عند التغيّر)
5	الملف الأكبر اسماً يفوز	`product_cost_history.sql` (`qualify row_number()`)
6	SCD2 و`valid_to` حصري وتقريب لخانتين	`product_cost_history.sql`
7	شروط السطر المحسوب (delivered، كمية > 0، تاريخ الرياض ≤ `2026-08-31`)	`weekly_margin.sql` و`stg_orders.sql`
8	إيراد السطر `round(qty × price − discount, 2)`	`stg_order_items.sql`
9	تكلفة السطر من النسخة السارية، وغير المُكلَّف يبقى في الإيراد	`weekly_margin.sql`
10	المدينة من الفرع والتصنيف من المنتج وأسبوع ISO من تاريخ الأوردر	`weekly_margin.sql`

6. مشاكل البيانات التي وُجدت وعولجت

المشكلة	أين	العدد	المعالجة
فاصلة عشرية `9,49`	W38	126 من 160	تحويلها إلى نقطة
أرقام عربية-هندية `٩٫٤٩`	W38	34 من 160	تحويل الأرقام والفاصل

علامة BOM	رأس W38	ملف واحد	القراءة بـ `utf-8-sig`
تغيير اسم عمود التكلفة	W39	160	`coalesce` للعمودين
تكلفة صفر أو سالبة	W39	4	حجر `invalid_cost`
SKU لا تبيعه قافلة	W39	3	حجر `unknown_sku`
مسافات في آخر SKU	products	2	`trim` في كل مقارنة
كمية ≥ 0	order_items	142 (129 في أوردرات مُسلَّمة)	استبعاد
أوردرات غير مُسلَّمة	orders	5,347 من 45,766	استبعاد
أوردرات قبل `2026-08-31` بتوقيت الرياض	orders	69,998 سطراً مُسلَّماً	استبعاد

7. القرارات التصميمية ولماذا

- **إعادة بناء raw في كل تشغيل:** أبسط طريقة تضمن عدم التكرار، والمصدر هو الحقيقة دائماً. ثمنها بطء مع بيانات ضخمة (انظر القسم 10).
- **قراءة SQLite بـ Python لا بإضافة DuckDB:** الإضافة قد تحتاج تنزيلاً من الإنترنت وبيئة المصحّح قد لا تسمح بذلك.
- **الـ dedupe بعد الحجر:** المهمة تقول إن الصفوف المحجورة «تُتجاهل في كل مكان آخر»، فالملف الأخير يفوز بين الصفوف **الصالحة** فقط. حالة نادرة لم تحددها المهمة، ولا أثر لها على بيانات التجربة.
- **snapshot واحد لكل تحديث:** استبدال الجدول يتم في commit واحد، فيكون «الـ snapshot السابق» هو الحالة الكاملة قبل التصحيح لا حالة وسيطة فارغة.
- **الكتابة فقط عند اختلاف المحتوى:** هذا ما يجعل إعادة التشغيل «لا تغيّر شيئاً» فعلاً.
- **أسبوع ISO بدالة `week()` في DuckDB:** تعطي أسبوع ISO 8601 (الأسبوع يبدأ الاثنين).

8. الاختبارات (34 اختبار dbt)

الاختبار	على	يحمي من
`unique` + `not_null`	معرّفات المنتجات والأوردرات والـ SKU	SKU مكرر يضاعف الأسطر في الربط
`not_null`	تاريخ الأوردر، التكلفة، `valid_from`، `lines`، `iso_week`، `reason`، `revenue_sar`	قيم فارغة تتسرب
`relationships`	sku في المنتجات ← sku في التاريخ	نسخة تكلفة لمنتج لا تبيعه قافلة

`accepted_values`	سبب الحجر	سبب غير / `unknown_sku` `invalid_cost`
`history_one_current_per_sku`	silver	نسخة حالية صفر أو اثنتان
`history_versions_are_contiguous`	silver	تداخل أو فجوة بين النسخ، وتكرار (sku, valid_from)
`history_new_version_only_on_change`	silver	نسخة جديدة بنفس التكلفة، أو تكلفة غير موجبة
`margin_grain_is_unique`	gold	صف مكرر لنفس أسبوع/مدينة/تصنيف
`margin_adds_up`	gold	هامش لا يساوي المُكلَّف − التكلفة
`margin_covers_every_eligible_line`	gold	الربط الزمني يُضيّع أو يكرّر أسطراً
`quarantine_rows_are_not_in_history`	silver	صف محجور يتسرب إلى التاريخ

9. كيف نتأكد أن الأرقام صحيحة

1. **اختبارات dbt:** تحمي سلامة الجداول (لا تكرار، لا تداخل، لا فقدان أسطر).
2. **`check.py` (22/22):** يفحص الشكل والاتساق: الأعمدة، قواعد SCD2، أن الهامش يجمع، وأن كل الأسطر المؤهلة مغطاة. **لا يعرف** إن كانت التكلفة نفسها صحيحة.
3. **`reference_check.py`:** يعيد حساب الجداول الثلاثة من الملفات الخام بـ Python عادي دون أي SQL ويقارنها بجداول Iceberg. النتيجة: 0 اختلاف على بيانات التجربة وعلى seed آخر (11)، قبل التصحيح وبعده.
4. **`spot_check.py`:** يتتبع SKU واحداً وسطراً واحداً من الملف الخام إلى الجدول النهائي لتراجعه بعينك.

> **حدّ صريح:** الحساب المستقل كُتب بنفس فهم القواعد، فهو يثبت أن الـ SQL يطبّق القواعد كما فُهمت، لا أن الفهم نفسه مطابق لقصد واضع المهمة. لذلك يُنصح بمراجعة القواعد العشر في القسم 5 يدوياً.

10. ما سيتغير مع نمو البيانات

مع بيانات أكبر 100 ضعفاً سيصبح الاختناق الأول هو إعادة بناء raw كاملاً وكتابة كامل الجدول إلى Iceberg. الحل المقترح: تحميل تزايدي (الملفات الجديدة فقط والأوردرات بحسب `updated_at`)، وجعل `weekly_margin` موديل dbt تزايدياً يعيد حساب الأسابيع المتأثرة بسعر جديد أو مصحَّح فقط، وتقسيم جدول Iceberg بحسب `iso_week`. ومع ملفات أسعار يومية يجب استبدال قاعدة «اسم الملف الأخير» بوقت استلام صريح لكل ملف.